```
import mlflow
from mlia_utils import mlflow_funcs

experiment_path = f"/Users/{current_user}/intel-clf-training_action"
mlflow_funcs.mlflow_set_experiment(experiment_path)
log_path = f"{volume_file_path}/intel_image_clf/intel_training_logger_board"
!mkdir {log_path}
```

Figure 6.25 – Defining experiment and log paths

Rather than working our way through the code top-to-bottom, we find it helpful to start by explaining the main training function `train_distributed`. This function contains the parameters we pass to the `pl.Trainer` to initialize and fit our trainer. *Figure 6.26* shows the code that you will find towards the end of the notebook in cell 12.

```
def train_distributed(max_epochs: int = 1, strategy: str = "auto"):
    # import logging
    # logging.basicConfig(level=logging.DEBUG)

    # this is required if you want to log with MlFlow
    # if you do not have access to a Token, simply save models under a checkpoint and log them back
    # Track your loss with the Tensoflow Board
    os.environ['DATABRICKS_HOST'] = db_host
    os.environ['DATABRICKS_TOKEN'] = db_token
    torch.set_float32_matmul_precision("medium")

    checkpoint_callback = pl.callbacks.ModelCheckpoint(…
    )
    early_stop_callback = pl.callbacks.EarlyStopping(…
    )

    tqdm_callback = pl.callbacks.TQDMProgressBar(…
    )
    # Initialize a trainer
    trainer = pl.Trainer(…
    )

    print(f"Global Rank: {trainer.global_rank}")
    print(f"Local Rank: {trainer.local_rank}")
    print(f"World Size: {trainer.world_size}")

    dm = DeltaDataModule()
    model = LitCVNet(num_classes=6)
    #trainer.fit(model, dm)
    trainer.fit(model, train_dataloaders=dm.train_dataloader(), val_dataloaders=dm.val_dataloader())
    print("Training done!")
```

Figure 6.26 – Defining training function